



Bakalářský projekt

Možnosti využití mikrokontroleru řady STM32F v součinnosti s wifi modulem ESP32

Studijní program:

Studijní obor:

Autor projektu:

Vedoucí projektu:

B0613A140005AI – Aplikovaná informatika

B0613A140005 – Informační technologie

Daniel Kňourek

Ing. Miroslav Holada, Ph.D.

Ústav informačních technologií a elektroniky





Zadání bakalářského projektu

Možnosti využití mikrokontroleru řady STM32F v součinnosti s wifi modulem ESP32

Jméno a příjmení:

Daniel Kňourek

Osobní číslo:

M24000015

Studijní program:

B0613A140005AI – Aplikovaná informatika

Studijní obor (specializace):

B0613A140005 – Informační technologie

Zadávací katedra:

Ústav informačních technologií a elektroniky

Akademický rok:

2022-2023/Květen 2026

Zásady pro vypracování:

1. Seznamte se s modulem Nucleo s obvodem STM32F746 a s modulem ESP32.
2. Navrhněte propojení obou modulů přes UART tak, aby kontrolér STM32 pracoval v reálném čase na zvolené úloze a ESP32 zajišťovalo webové uživatelské rozhraní pro konfiguraci úlohy.
3. Navrhněte a odlaďte ukázkový program tak, aby byly názorně demonstrovány možnosti zvoleného řešení.



Rozsah grafických prací:
Forma zpracování projektu:
Jazyk projektu:

Dle potřeby dokumentace
Elektronická
Čeština

Seznam odborné literatury:

- [1] NOVÁK, Petr. Mobilní roboty: pohony, senzory, řízení. 1. vyd. Praha: BEN - technická literatura, 2005. ISBN80-7300-141-1.
- [2] www.st.com
- [3] <https://www.espressif.com>

Vedoucí projektu:

Ing. Miroslav Holada, Ph.D.
Ústav informačních technologií a elektroniky

Datum zadání projektu:

2022-2023

Předpokládaný termín odevzdání:

Květen 2026

doc. Ing. Josef Černohorský, Ph.D.
děkan

L.S.

doc. Ing. Josef Chaloupka, Ph.D.
vedoucí ústavu

Prohlášení

Prohlašuji, že jsem svůj bakalářský projekt vypracoval samostatně jako původní dílo s použitím uvedené literatury a na základě konzultací s vedoucím mého projektu.

Jsem si vědom toho, že na můj projekt se plně vztahuje zákonč. 121/2000 Sb., o právu autorském, zejména § 60 – školní dílo.

Beru na vědomí, že Technická univerzita v Liberci nezasahuje do mých autorských práv užitím mého projektu pro vnitřní potřebu Technické univerzity v Liberci.

Užiji-li projekt nebo poskytnu-li licenci k jeho využití, jsem si vědom povinnosti informovat o této skutečnosti Technickou univerzitu v Liberci; v tomto případě má Technická univerzita v Liberci právo ode mne požadovat úhradu nákladů, které vynaložila na vytvoření díla, až do jejich skutečné výše.

Současně čestně prohlašuji, že text elektronické podoby projektu vložený do IS STAG se shoduje s textem tištěné podoby projektu.

Beru na vědomí, že můj projekt bude zveřejněn Technickou univerzitou v Liberci v souladu s § 47b zákona č. 111/1998 Sb., o vysokých školách a o změně a doplnění dalších zákonů (zákon o vysokých školách), ve znění pozdějších předpisů.

Jsem si vědom/a následků, které podle zákona o vysokých školách mohou vyplývat z porušení tohoto prohlášení.

20. května 2026

Daniel Kňourek

Možnosti využití mikrokontroleru řady STM32F v součinnosti s wifi modulem ESP32

Abstrakt

Tento bakalářský projekt se zabývá návrhem, realizací a vyhodnocením vestavěného systému se dvěma mikrokontroléry, který efektivně kombinuje výkonný čip řady STM32F7 s bezdrátovým modulem ESP32 za účelem striktního oddělení nízkoúrovňového řízení v reálném čase od asynchronní zátěže spojené se sítovou komunikací. V rámci projektu byl navržen a vyroben vlastní rozšiřující hardwarový modul (shield) integrující obvod regulované analogové zpětné smyčky. Pro obousměrnou komunikaci přes rozhraní UART byl navržen vlastní binární protokol využívající formát Protocol Buffers s dvoufázovým přenosem rámců. Na straně modulu ESP32 byl v prostředí ESP-IDF vytvořen vestavěný webový server s responzivním uživatelským rozhraním, které kombinuje bezstavovou architekturu REST API s plně duplexním streamováním dat v reálném čase pomocí protokolu WebSockets. Funkčnost celého řetězce byla ověřena třemi testovacími programy.

Klíčová slova

STM32, ESP32, UART, Protocol Buffers, WebSockets, FreeRTOS, vestavěné systémy

Potential of Using the STM32F Series Microcontroller in Conjunction with the ESP32 Wi-Fi Module

Abstract

This bachelor's project addresses the design, implementation, and evaluation of an embedded system with two microcontrollers that combines a powerful STM32F7 series chip with an ESP32 wireless module to strictly separate low-level, real-time control from the asynchronous load associated with network communication. As part of the project, a custom hardware expansion module (shield) integrating a regulated analogue feedback loop circuit was designed and manufactured. For bidirectional communication over the UART interface, a custom binary protocol was designed using the Protocol Buffers format with a two-phase frame transmission. On the ESP32 module side, an embedded web server with a responsive user interface was created within the ESP-IDF environment, combining a stateless REST API architecture with full-duplex, real-time data streaming via the WebSockets protocol. The functionality of the entire system chain was verified using three test programs.

Keywords

STM32, ESP32, UART, Protocol Buffers, WebSockets, FreeRTOS, embedded systems

Obsah

1	Úvod.....	11
2	Teoretická část	12
2.1	Použité mikrokontroléry a vývojové desky	12
2.1.1	Vývojová deska STM32F746G-DISCOVERY	12
2.1.2	Modul ESP32 DevKit v1	13
2.2	Sériová komunikace a protokoly	14
2.2.1	Sériová linka UART a princip jejího fungování	14
2.2.2	Fyzické zapojení a propojení modulů	15
2.2.3	Způsoby zpracování dat	15
2.3	Koncepty softwarového vývoje pro mikrokontroléry.....	16
2.3.1	Architektura bez operačního systému	16
2.3.2	Operační systémy reálného času	16
2.4	Datová serializace a komunikace.....	16
2.5	Vývojová a softwarová prostředí	17
2.5.1	Prostředí pro platformu STM32	18
2.5.2	Prostředí pro platformu ESP32	18
2.5.3	Unifikace vývoje pomocí kontejnerizace.....	18
3	Praktická část	19
3.1	Hardwarový návrh a propojení modulů	19
3.1.1	Sériové rozhraní UART	19
3.1.2	Indikační obvod se stavovými LED.....	20
3.1.3	Obvod regulované analogové smyčky	21
3.1.4	Finální hardwarové sestavení.....	22
3.2	Návrh a struktura komunikačního protokolu	23
3.2.1	Mechanismus dvoufázového přenosu dat	23
3.2.2	Řešené technické výzvy při asynchronním přenosu	24
3.2.3	Definiční schéma zpráv.....	24
3.3	Softwarová implementace mikrokontroléru STM32	25
3.3.1	Hlavní programová smyčka a koordinace úloh	25
3.3.2	Řízení sériového rozhraní a zpracování rámců	26
3.3.3	Aplikační manažer	26
3.4	Softwarová implementace modulu ESP32	27
3.4.1	Inicializace systému a správa Wi-Fi konektivity	27
3.4.2	Sériová komunikace, asynchronní příjem a validace.....	27

3.4.3	Webový server pro vzdálené řízení systému.....	28
	Webová aplikace a uživatelské rozhraní	29
3.4.4	Síťová konektivita a inicializace připojení	29
3.4.5	Kombinovaný komunikační model rozhraní	30
3.4.6	Vizuální struktura a globální prvky rozhraní	31
3.5	Ukázkové programy a testování propustnosti.....	31
3.5.1	Program 1: Verifikace komunikačního řetězce	31
3.5.2	Program 2: Analýza propustnosti.....	32
3.5.3	Program 3: Komplexní integrace	33
4	Diskuze	35
4.1	Překonané výzvy.....	35
4.1.1	Diagnostika síťové vrstvy a limity HTTP/1.1.....	35
4.1.2	Správa prostředků a úniky WebSocketů na mobilních zařízeních	35
4.1.3	Hardwarové interference nepájených spojů.....	36
4.2	Možnosti budoucího rozšíření	36
5	Závěr.....	37
	Použitá literatura.....	38
	Přílohy	40
A	Digitální archiv projektu	40

Seznam obrázků

Obrázek 1 – Vývojová deska 32F746GDISCOVERY	12
Obrázek 2 – Rozložení pinů vývojové desky 32F746GDISCOVERY	13
Obrázek 3 – Detailní schéma pinů a sběrnic modulu DOIT ESP32 DEVKIT v1	14
Obrázek 4 – Fyzické zapojení křížené sériové linky UART mezi deskami ESP32 a STM32. 20	
Obrázek 5 – Fotografie finálního zapojení systému – boční pohled na osazený shield s ESP32 a STM32.....	22
Obrázek 6 – Fotografie finálního zapojení systému – celkový pohled shora na rozmístění indikačních a regulačních prvků	23
Obrázek 7 – Bezpečnostní upozornění webového prohlížeče při přístupu na lokální IP adresu	30
Obrázek 8 – Postup pro vynucení pokračování na webové rozhraní v pokročilém nastavení. 30	
Obrázek 9 – Indikátor aktivity systému a přepínač barevného tématu aplikace.....	31
Obrázek 10 – Diagnostický log událostí (SystemConsole) na straně webového klienta.....	31
Obrázek 11 – Ovládací panel Programu 1 (Alive ping) ve webovém rozhraní	32
Obrázek 12 – Ovládací rozhraní a tabulka živých statistik propustnosti v Programu 2	32
Obrázek 13 – Volba konstantního DC signálu.....	33
Obrázek 14 – Konfigurace parametrů sinusového signálu.....	34
Obrázek 15 – Nastavení vzorkování a velikosti rámce	34
Obrázek 16 – Ovládací prvky pro zobrazení grafu	34
Obrázek 17 – Reálná vizualizace streamovaných ADC dat v reálném čase se zachycením manuální regulace potenciometru	35

Seznam schémat

Schéma 1 – Schéma indikačního obvodu se třemi stavovými LED diodami	21
Schéma 2 – Schéma obvodu regulované analogové zpětné smyčky	21

Seznam tabulek

Tabulka 1 – Přehled komunikačního rozhraní (API) na modulu ESP32	29
--	----

Seznam zdrojových kódů

Zdrojový kód 1 – Definiční schéma komunikačního protokolu v souboru <i>messenger.proto</i> ..	25
Zdrojový kód 2 – Konfigurace Wi-Fi rozhraní v ESP-IDF projektu	27

Seznam zkratek

UART	Universal AsynchronousReceiver-Transmitter
MCU	Microcontroller Unit
GPIO	General-Purpose Input/Output
HAL	Hardware AbstractionLayer
ISR	InterruptServiceRoutine
IDE	IntegratedDevelopment Environment
ADC	Analog-to-Digital Converter
DAC	Digital-to-Analog Converter
DMA	Direct MemoryAccess
PWM	Pulse-WidthModulation
RTOS	Real-Time OperatingSystem
FIFO	First In, First Out

1 Úvod

Vývoj vestavěných (embedded) systémů je dnes úzce spjat s potřebou bezdrátové konektivity a vzdálené správy. Mikrokontroléry řady STM32F vynikají vysokým výpočetním výkonem, spolehlivostí a bohatou výbavou periferií pro přesné řízení úloh v reálném čase. (STMicroelectronics, 2026a) Často jim však chybí nativní síťová podpora. Tuto mezeru efektivně vyplňují moduly ESP32 s integrovanou Wi-Fi konektivitou, které se díky svému výkonu a dostupnosti staly standardem v oblasti internetu věcí (IoT). (Espressif Systems, 2025)

K volbě tohoto tématu mě vedl také blízký vztah k moderním technologiím a prvkům chytré domácnosti, jako jsou inteligentní žárovky nebo zásuvky, které sám aktivně využívám. Skutečnost, že jsou tato zařízení dnes již běžnou součástí našich životů, ve mně probudila hlubší zájem o to, jak tato technika funguje uvnitř. Tento projekt mi tak poskytl ideální příležitost nahlédnout pod kapotu systémů, se kterými se denně setkávám, a prozkoumat jejich chování a implementaci v reálné praxi.

Hlavním cílem tohoto bakalářského projektu je spojit silné stránky obou platforem do jednoho funkčního a robustního celku. Návrh systému izoluje nízkoúrovňové řízení v reálném čase na čipu STM32, aniž by byl procesor zatěžován asynchronní sítíovou komunikací. Tuto agendu přebírá modul ESP32, který slouží jako brána mezi lokálním hardwarem a uživatelem. Zajišťuje webové rozhraní, prostřednictvím kterého lze celý systém pohodlně konfigurovat z mobilního telefonu nebo počítače. V souladu se zadáním jsou obě odlišné architektury propojeny sériovým rozhraním UART. Klíčovou výzvou je návrh spolehlivého komunikačního protokolu pro efektivní obousměrný přenos dat, analýza propustnosti tohoto spojení a detekce jeho limitujících faktorů.

Text projektu je logicky strukturován do několika částí. Teoretická část se věnuje seznámení s využitým hardwarem, softwarovou architekturou a komunikačními protokoly nutnými pro spolupráci obou platforem. Praktická část podrobně popisuje fyzické zapojení a softwarovou implementaci komunikačního řetězce. V závěru jsou pak prostřednictvím odladěných ukázkových programů vyhodnoceny reálné možnosti a limity navrženého řešení v praxi.

2 Teoretická část

Teoretická část tohoto bakalářského projektu slouží jako ucelený odborný základ pro následnou praktickou realizaci. Cílem této kapitoly je detailně představit klíčové hardwarové komponenty, tedy použité mikrokontroléry a vývojové desky, a vysvětlit základní principy komunikačních rozhraní, softwarové architektury a vývojových prostředí, které byly k řešení použity.

Znalost těchto technických specifik je nezbytná pro pochopení limitů a možností celého navrhovaného distribuovaného systému. Právě volba vhodného hardwaru a pochopení jeho vnitřní architektury určuje, jak efektivně dokáže systém pracovat v reálném čase a jak spolehlivě dokáže přenášet data.

2.1 Použité mikrokontroléry a vývojové desky

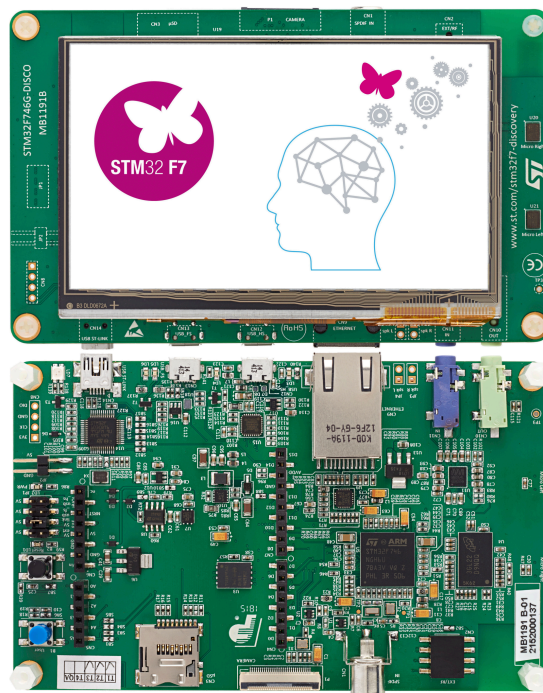
Pro realizaci tohoto projektu byly zvoleny dvě odlišné hardwarové platformy, z nichž každá plní specifickou roli. První platformou je výkonná vývojová deska od společnosti STMicroelectronics (STMicroelectronics, 2026a), která zajišťuje nízkoúrovňové řízení a výpočty. Druhou platformou je populární Wi-Fi modul postavený na čipu ESP32, jenž se stará o síťovou konektivitu a poskytuje uživatelské webové rozhraní. (Espressif Systems, 2026)

2.1.1 Vývojová deska STM32F746G-DISCOVERY

Jako hlavní výpočetní jednotka celého systému byla zvolena vývojová deska 32F746GDISCOVERY. Tato deska je osazena vysoce výkonným mikrokontrolérem STM32F746NG, jehož jádrem je 32bitová architektura Arm® Cortex®-M7. Tento procesor je schopen pracovat na frekvenci až 212 MHz, což mu poskytuje dostatečný výpočetní výkon pro komplexní úlohy prováděné v reálném čase.

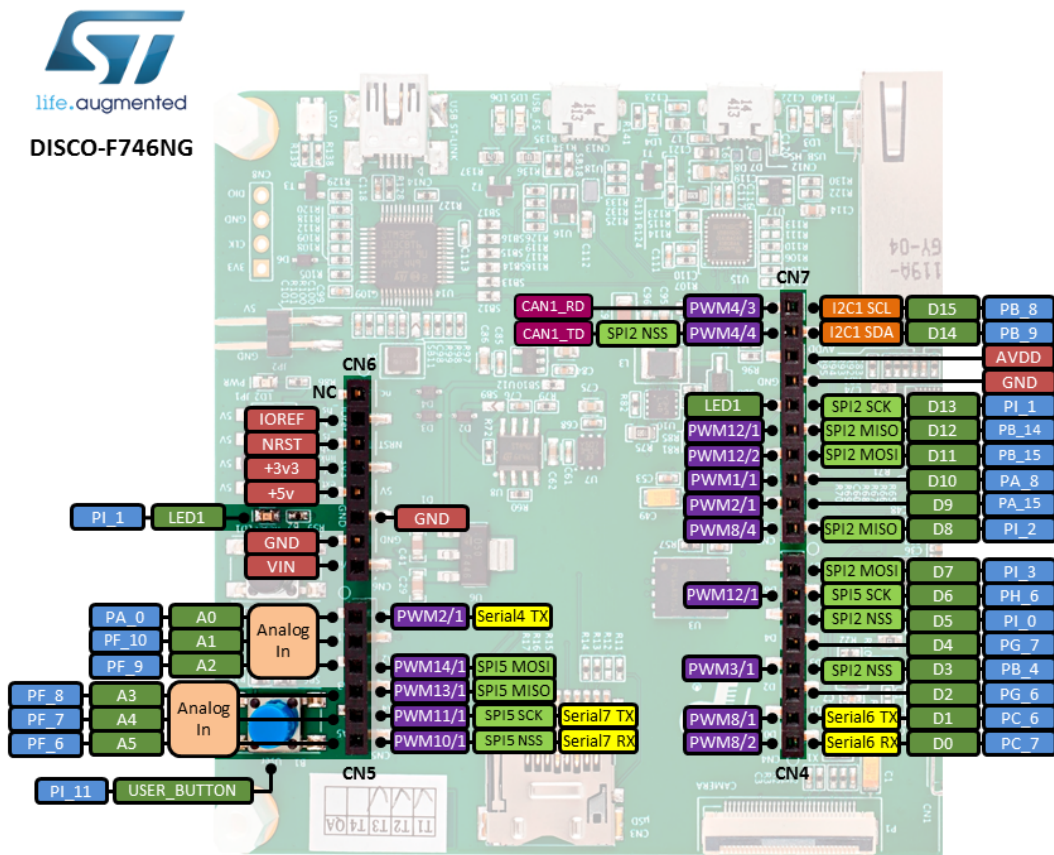
Z hlediska paměťové vybavy mikrokontrolér disponuje interní pamětí o velikosti 340 KB RAM a 1 MB paměti Flash. Pro náročnější aplikace, jako je například ukládání grafických prvků nebo obsáhlejších datových bufferů, deska nabízí také externí paměti v podobě 8 MB SDRAM a 16 MB Flash. Celkový vzhled desky je zachycen na Obrázku 1.

Kromě samotného výpočetního výkonu je deska bohatě vybavena perifériemi, což z ní činí univerzální nástroj pro vývoj embedded systémů. Výrazným prvkem je integrovaný 4,3palcový barevný LCD-TFT displej s kapacitní dotykovou vrstvou. Dále deska obsahuje konektory kompatibilní s rozložením pinů (pinoutem) ARDUINO® Uno V3, rozhraní Ethernet, audio



Obrázek 1 – Vývojová deska 32F746GDISCOVERY (STMicroelectronics, 2026a)

vstupy a výstupy, konektor pro kameru, USB OTG a slot pro paměťové karty microSD. Rozložení jednotlivých pinů a dostupných rozhraní podrobně ilustruje Obrázek 2.



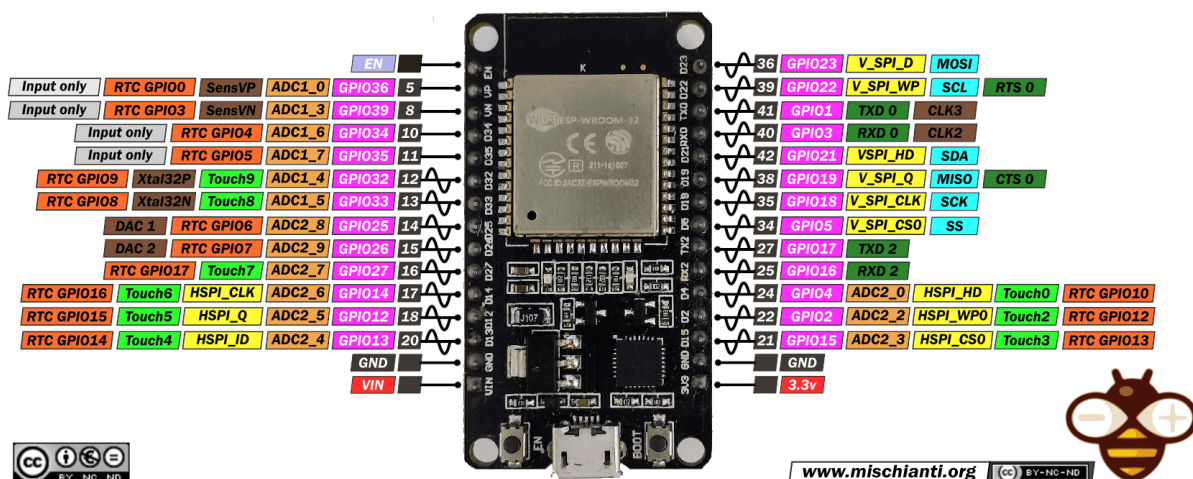
Obrázek 2 – Rozložení pinů vývojové desky 32F746GDISCOVERY
(Arm Ltd.)

2.1.2 Modul ESP32 DevKit v1

Pro zajištění bezdrátové konektivity a asynchronní komunikace s uživatelem byl využit modul ESP32, konkrétně ve variantě vývojové desky DOIT ESP32 DEVKIT v1. Srdcem této desky je výkonný čip ESP-WROOM-32, který integruje 32bitový dvoujádrový procesor pracující na frekvenci až 240 MHz.

Architektura tohoto čipu je navržena s primárním ohledem na zařízení typu Internet věcí (IoT). Modul nabízí 512 KB operační paměti RAM a 16 MB paměti Flash, což je plně dostačující pro běh operačního systému reálného času (RTOS) i pro hostování vlastního webového serveru.

ESP32 DEV KIT V1 PINOUT



Obrázek 3 – Detailní schéma pinů a sběrnic modulu DOIT ESP32 DEVKIT v1 (Mischianti, 2021)

Největší výhodou modulu ESP32 je jeho integrovaná podpora pro Wi-Fi a Bluetooth komunikaci přímo na čipu. Kromě síťových funkcí deska vyvádí širokou škálu periferních sběrnic, mezi které patří piny pro všeobecné použití (GPIO), pulzně šířková modulace (PWM), rozhraní USB OTG a především asynchronní sériová linka UART, která hraje klíčovou roli v propojovací topologii tohoto projektu. Jak je patrné z Obrázku 3, modul nabízí dostatečné množství hardwarových rozhraní v kompaktním formátu.

2.2 Sériová komunikace a protokoly

Pro výměnu dat mezi řídicím mikrokontrolérem STM32 a komunikačním modulem ESP32 bylo zvoleno sériové rozhraní UART. Cílem této kapitoly je vysvětlit teoretické principy tohoto protokolu, popsat realizované hardwarové propojení a analyzovat softwarové možnosti zpracování dat na straně mikrokontroléru.

2.2.1 Sériová linka UART a princip jejího fungování

UART (neboli univerzální asynchronní přijímač-vysílač) je standardní hardwarový komunikační protokol hojně využívaný pro asynchronní sériovou komunikaci. Na rozdíl od synchronních sběrnic, jako jsou SPI nebo I2C, nevyužívá UART společný hodinový signál k synchronizaci vysílače a přijímače zařízení. (Analog Devices, 2024)

Namísto hodinového signálu se obě komunikující strany musí předem nastavit na identickou přenosovou rychlost (tzv. baud rate), která udává počet přenesených bitů za jedinou sekundu. Aby přijímač dokázal spolehlivě detekovat, kdy začíná a kdy končí přenášená zpráva, jsou samotná data zapouzdřena do takzvaného datového rámce. Rámec je uvozen start bitem, za kterým následuje datový obsah (tzv. užitečné zatížení neboli payload), a celý přenos je zakončen jedním nebo více stop bity. (Analog Devices, 2024)

Zásadní výhodou sběrnice UART je její hardwarová jednoduchost a nenáročnost. Pro realizaci plně duplexní (obousměrné a současně) komunikace mezi dvěma mikrokontroléry postačují pouze tři fyzické vodiče (Analog Devices, 2024):

- **TX (Transmit):** Vysílací linka, přes kterou zařízení odesílá svá data do okolí.
- **RX (Receive):** Příjímací linka, na které zařízení přijímá data od protistrany.
- **GND (Ground):** Společná zem sloužící ke sjednocení referenčních napětových úrovní obou zařízení.

2.2.2 Fyzické zapojení a propojení modulů

Pro navázání úspěšné komunikace a realizaci plně duplexního přenosu je z hlediska topologie této sběrnice nutné propojit vysílací pin (Transmit – TX) jednoho zařízení s přijímacím pinem (Receive – RX) druhého zařízení a analogicky zajistit i opačný směr (tzv. křížené zapojení). (STMicroelectronics, 2026b)

Zásadním hardwarovým požadavkem je rovněž přímé elektrické propojení společné země (GND) obou systémů. Tímto spojením je zajištěna nezbytná stabilizace přenosového signálu a sjednocení referenčních napětových úrovní, vůči kterým obě odlišné hardwarové architektury spolehlivě vyhodnocují logické stavy na lince. Tato popsaná hardwarová konfigurace představuje nezbytnou fyzickou vrstvu pro následný přenos binárních serializovaných dat a tvoří obecný teoretický základ pro konkrétní pinové mapování obou mikrokontrolérů, které je detailně popsáno a vyobrazeno v praktické části projektu. (STMicroelectronics, 2026b)

2.2.3 Způsoby zpracování dat

Mít fyzicky propojenou linku je pouze prvním krokem; stejně důležitý je i způsob, jakým mikrokontrolér přijatá data zpracovává. V praxi embedded systémů se nejčastěji setkáváme se třemi softwarovými přístupy. Nejprimitivnější metodou je blokující dotazování (tzv. polling), kdy procesor (CPU) neustále kontroluje registry linky, zda nepřišla nová data. To je však vysoce neefektivní, protože procesor ztrácí čas, který by mohl využít k výpočtům. (EmbeddedExpertIO, 2025)

Z toho důvodu se v reálných asynchronních úlohách nasazují efektivnější hardwarové principy (EmbeddedExpertIO, 2025):

- **Přerušení (ISR):** Při nasazení přerušení nemusí procesor linku aktivně hlídat. V momentě, kdy rozhraní UART úspěšně detekuje příchozí bajt, vygeneruje vnitřní hardwarový signál. Tento signál okamžitě pozastaví aktuální činnost procesoru, který následně „odskočí“ vykonat obslužnou rutinu (ISR), přesune bajt do paměti a vrátí se ke své původní činnosti. Přestože je toto řešení neblokující a velmi oblíbené, má i svou nevýhodu. U vysokých přenosových rychlostí způsobuje neustálé vyvolávání přerušení (pro každý přijatý bajt) velkou výpočetní zátěž procesoru neboli tzv. overhead.
- **Přímý přístup do paměti (DMA):** Pro dosažení nejvyšší propustnosti se k lince UART připojuje takzvaný DMA řadič. Jedná se o specializovanou hardwarovou jednotku, která se stará o přesun dat z přijímacího registru sériové linky přímo do paměti RAM a to naprosto nezávisle na procesoru. Procesor pouze jednou DMA zkonfiguruje a poté se věnuje svým uživatelským programům a výpočtům. DMA následně upozorní procesor vyvoláním přerušení (např. Transfer Completecallback) až teprve ve chvíli, kdy

spolehlivě přesune celý očekávaný blok či rámec dat. Tento princip radikálně snižuje zátěž procesoru a uvolňuje výkon pro souběžné zpracování měřicích úloh (např. z generování plynulé sinusoidy či čtení senzorů).

2.3 Koncepty softwarového vývoje pro mikrokontroléry

Vývoj softwaru pro vestavěné (embedded) systémy vyžaduje zcela odlišný přístup než tvorba běžných počítačových aplikací. Vzhledem k omezenému výkonu, paměti a nutnosti reagovat na události v reálném čase se v praxi využívají dvě základní architektury. Tento projekt kombinuje obě z nich – mikrokontrolér STM32 funguje jako tzv. bare-metal aplikace, zatímco modul ESP32 využívá operační systém reálného času (RTOS).

2.3.1 Architektura bez operačního systému

- **Bare-metal přístup:** Vývoj softwaru běžícího přímo na hardwaru bez jakékoliv mezivrstvy operačního systému, což zajišťuje maximální výpočetní efektivitu a determinismus (Silicon Signals Pvt. Ltd., 2024). Tento přístup je využit u mikrokontroléru STM32.
- **Superloop:** Architektura bare-metal aplikací založená na nekonečné programové smyčce, která sekvenčně a neblokujícím způsobem vykonává procesní funkce (Silicon Signals Pvt. Ltd., 2024). Asynchronní události jsou zachytávány pomocí hardwarových přerušování (ISR), která s hlavní smyčkou komunikují skrze systém stavových příznaků (flagů).

2.3.2 Operační systémy reálného času

- **RTOS:** Operační systém reálného času integrovaný do vývojového frameworku ESP-IDF. Pro modul ESP32 je využita modifikovaná verze systému FreeRTOS, která je plně přizpůsobena pro správu více souběžných úloh a síťových operací. (Espressif Systems, 2021)
- **Úloha / Vlákno:** Samostatný, izolovaný blok kódu (vlákno) v rámci FreeRTOS, který disponuje vlastní prioritou a vyhrazenou pamětí. Plánovač operačního systému mezi těmito úlohami velmi rychle přepíná; specifika ESP-IDF navíc umožňují jednotlivé úlohy fixovat na konkrétní procesorové jádro (tzv. corepinning) pro dosažení reálného paralelního běhu na dvoujádrovém čipu ESP32. (Espressif Systems, 2021)
- **Fronta:** Základní datová struktura typu FIFO určená pro bezpečnou asynchronní komunikaci a předávání zpráv nebo datových struktur mezi nezávislými úlohami (vlákny) bez rizika vzniku kolizí a souběhu nad sdílenou pamětí. (Espressif Systems, 2021)

2.4 Datová serializace a komunikace

Pro efektivní a spolehlivou výměnu informací mezi dvěma odlišnými hardwarovými architekturami mikrokontrolérů je nezbytné definovat rigidní způsob reprezentace přenášených dat. V oblasti embedded systémů hraje klíčovou roli datová serializace, což je proces převodu strukturovaných aplikačních objektů, struktur a proměnných do kompaktního binárního proudu bajtů, který je vhodný pro přenos po fyzické sériové lince (GOOGLE PROTOBUF, 2026). Na

cílovém zařízení pak asynchronně probíhá opačný proces, takzvaná deserializace, která binární proud rekonstruuje zpět do podoby původních programových struktur.

Jako hlavní serializační mechanismus byl v tomto projektu zvolen binární protokol vyvinutý společností Google s názvem ProtocolBuffers, zkráceně Protobuf (GOOGLE PROTOBUF, 2026). Tento mechanismus je zcela nezávislý na použitém programovacím jazyce či hardwarové platformě. Oproti standardním textovým formátům, jako jsou JSON nebo XML, generuje Protobuf výrazně menší datový objem (overhead) a jeho kódování i dekódování vyžaduje řádově nižší výpočetní výkon. To představuje zásadní výhodu pro mikrokontroléry disponující omezenou pamětí RAM a limitovanými procesorovými cykly.

Vzhledem k tomu, že vývojová prostředí obou platform pracují primárně s jazykem C, bylo nutné pro realizaci zvolit specifické knihovny implementující tento standard. Na straně Wi-Fi modulu ESP32, v rámci oficiálního frameworku ESP-IDF, je využita knihovna protobuf-c (PROTOBUF-C, 2026). Pro řídicí mikrokontrolér STM32F746NG byla naopak zvolena vysoce úsporná knihovna nanopb. Tato knihovna je navržena speciálně pro systémy s kritickým nedostatkem paměti, přičemž veškeré operace kódování a dekódování provádí s pevnou a staticky alokovanou velikostí bufferů, což zcela eliminuje riziko fragmentace paměti RAM.

Přenos zpráv po asynchronní sériové lince UART vyžaduje návrh specifické struktury datového rámce. To mimo jiné obnáší i kontrolní součet CRC, který slouží k detekci chyb a zahození poškozených dat. Více v praktické části.

Užitečné zatížení pak nese samotný operační obsah. Využitím klíčového slova *oneof* v definičním souboru protokolu je dosaženo vysoké flexibility, kdy jeden univerzální rámec může podle potřeby přenášet buď konfigurační instrukce pro programy, nebo různé typy naměřených senzorových dat podle specifikace standardu ProtocolBuffers. (GOOGLE PROTOBUF, 2026)

Zatímco nízkourovňová komunikace mezi oběma čipy spoléhá na binární formát Protobuf přenášený přes rozhraní UART, komunikace mezi uživatelským prohlížečem a modulem ESP32 probíhá na bezdrátové síťové vrstvě Wi-Fi. Pro zprostředkování uživatelského rozhraní slouží webový server integrovaný v ESP32, který hostuje aplikaci klienta.

Z hlediska síťové komunikace využívá tento systém hybridní přístup. Pro odesílání jednorázových konfiguračních příkazů slouží bezstavová architektura REST API nad standardním protokolem HTTP. Aby však bylo souběžně zajištěno plynulé, obousměrné a nízkenergetické streamování naměřených dat v reálném čase, využívá se pro tento datový kanál plně duplexní síťový protokol WebSockets (MDN Web Docs, 2025). Toto rozdělení úloh prokazatelně eliminuje vysokou režii spojenou s neustálým dotazováním a opakovaným navazováním HTTP relací.

2.5 Vývojová a softwarová prostředí

Pro úspěšný vývoj, kompilaci a následné ladění firmwaru obou mikrokontrolérů bylo nezbytné zvolit a správně nakonfigurovat moderní softwarové nástroje. Cílem této kapitoly je představit integrovaná vývojová prostředí, specializovaná rozšíření a virtualizační technologie, které umožnily sjednotit vývojový proces a zajistit plnou kompatibilitu celého projektu.

2.5.1 Prostředí pro platformu STM32

Prvním pilířem softwarové výbavy je integrované vývojové prostředí STM32CubeIDE od společnosti STMicroelectronics, které slouží jako oficiální nástroj pro práci s mikrokontroléry řady STM32 (STMICROELECTRONICS, 2026c). Toto prostředí je postaveno na platformě Eclipse a integruje v sobě pokročilý editor kódu, kompilátor GCC a nástroje pro debugování.

Zásadní výhodou STM32CubeIDE je přítomnost vestavěného grafického konfigurátoru. Ten vývojáři umožňuje vizuálně navrhovat přiřazení hardwarových pinů, spravovat strom hodinových frekvencí a automaticky generovat inicializační kód v jazyce C s využitím hardwarové abstrakční vrstvy HAL (STMICROELECTRONICS, 2026c). Tento přístup radikálně urychluje úvodní konfiguraci periférií, jako je rozhraní UART a DMA řadič.

2.5.2 Prostředí pro platformu ESP32

Pro tvorbu aplikací na platformě ESP32 bylo zvoleno vývojové prostředí Visual Studio Code od společnosti Microsoft (MICROSOFT, 2026b). Jedná se o vysoce všestranný a modulární editor kódu, jehož schopnosti lze flexibilně rozšiřovat.

Pro účely tohoto projektu je stěžejní oficiální plugin ESP-IDF Extension, který poskytuje kompletní rozhraní k nativnímu frameworku od společnosti Espressif Systems (Espressif Systems, 2026d). Toto rozšíření integruje grafické konfigurační menu projektu, nástroje pro automatickou kompilaci, flashování firmwaru do paměti čipu a monitorování sériového výstupu přes USB rozhraní v reálném čase.

2.5.3 Unifikace vývoje pomocí kontejnerizace

Aby se předešlo klasickým problémům nekompatibility vývojových nástrojů, odlišných verzí knihoven a toolchainů mezi různými operačními systémy na straně vývojáře, byla do procesu začleněna platforma Docker. Docker umožňuje vytvořit izolované, nezávislé a přesně replikovatelné prostředí zvané kontejner. Tento kontejner v sobě nese definované verze kompilátorů a závislostí frameworku ESP-IDF, aniž by jakkoliv ovlivňoval hostitelský operační systém (Docker Inc., 2026).

Propojení mezi editorem Visual Studio Code a běžícím izolovaným kontejnerem zajišťuje specializovaný plugin DevContainers (MICROSOFT, 2026a). Tento doplněk dokáže automaticky detekovat konfigurační soubory projektu, spustit odpovídající Docker kontejner a transparentně do něj připojit celé vývojové prostředí editoru (MICROSOFT, 2026c). Díky tomuto unifikovanému přístupu k vývoji embedded systémů je na lokálním stroji zapotřebí pouze VS Code, Git a Docker, což zajišťuje stoprocentní přenositelnost a okamžitou funkčnost projektu na jakémkoliv počítači.

3 Praktická část

Praktická část tohoto bakalářského projektu popisuje konkrétní návrh, hardwarovou komplekci, strukturu komunikačního protokolu a softwarovou implementaci celého distribuovaného systému. Vývoj se opírá o propojení vestavěných platforem STM32 a ESP32 za účelem vytvoření stabilního asynchronního řetězce pro sběr dat a vzdálenou správu v reálném čase.

3.1 Hardwarový návrh a propojení modulů

Základem fyzické vrstvy celého systému je elektrické propojení obou řídicích mikrokontrolérů a realizace pomocných periférií na společné prototypové desce (shieldu = specifický typ desky plošných spojů (PCB), která je navržena k přímé mechanické a elektrické nadstavbě nad základní vývojovou desku mikrokontroléru (HWLab FIT ČVUT, 2016)). Tyto periferní obvody slouží k optické signalizaci interních stavů systému a k simulaci měřeného analogového subsystému pomocí regulované zpětné vazby.

3.1.1 Sériové rozhraní UART

Propojení mezi modulem ESP32 a deskou STM32 vyžaduje z hlediska sběrnice UART celkem tři elektrické spoje, které zajišťují společnou zem a křížené sériové linky pro duplexní přenos dat.

- **Společná zem:** První spoj propojuje zemní piny (GND) obou desek, což je nezbytné pro sjednocení referenčních napěťových úrovní v obou hardwarových architekturách.
- **Směr ESP -> STM:** Druhý spoj propojuje přijímací pin RXD 2 (GPIO16) na modulu ESP32 s vysílacím pinem D1 (Serial6 TX) na straně desky STM32.
- **Směr STM -> ESP:** Třetí spoj zajišťuje opačný směr komunikace a spojuje vysílací pin TXD 2 (GPIO17) na modulu ESP32 s přijímacím pinem D0 (Serial6 RX) na vývojové desce STM32.

Fyzické trasování signálových linek a celkové schéma kříženého zapojení obou platforem je podrobně specifikováno na Obrázku 4.

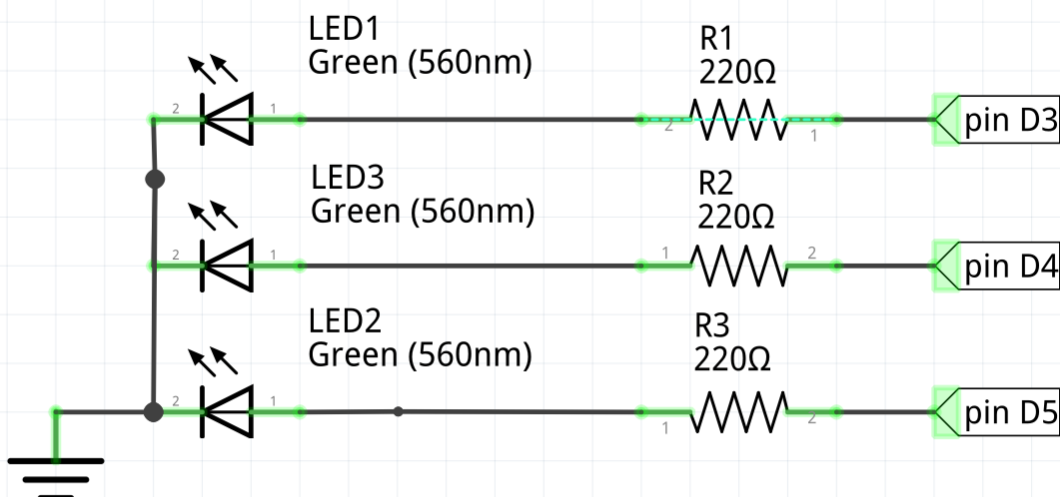


Schéma 1 – Schéma indikačního obvodu se třemi stavovými LED diodami
(vlastní zpracování, fritzing.org)

3.1.3 Obvod regulované analogové smyčky

Navržená a realizovaná regulovaná analogová zpětná smyčka slouží k převodu, generování a následnému měření signálů mezi výstupy a vstupy vývojové desky STM32F746G-DISCO. Obvod funguje jako hardwarová zpětná vazba, která převádí digitální software generovaný signál na analogovou veličinu. Zapojení se skládá ze dvou hlavních funkčních bloků: z integračního RC filtru dolní propusti a z lineárního potenciometru.

Při návrhu a realizaci senzorických subsystémů pro vestavěné systémy je klíčovým úkolem zajištění imunity vůči vnějším vlivům a elektromagnetickému šumu, který by mohl zkreslit měřené veličiny. Jak ve své publikaci uvádí (Novák, 2005), vhodným hardwarovým filtrováním a úpravou signálu je nezbytné docílit vyšší odolnosti přijímacího subsystému vůči rušivým vlivům okolního prostředí. Tento princip potlačení vysokofrekvenčního šumu a stabilizace referenční úrovně byl v projektu aplikován formou integračního RC filtru dolní propusti.

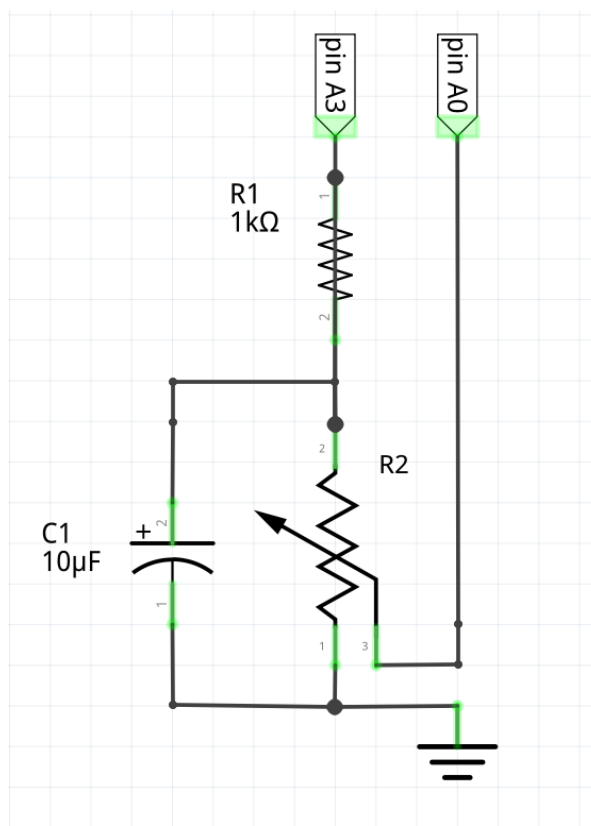


Schéma 2 – Schéma obvodu regulované analogové zpětné smyčky
(vlastní zpracování, fritzing.org)

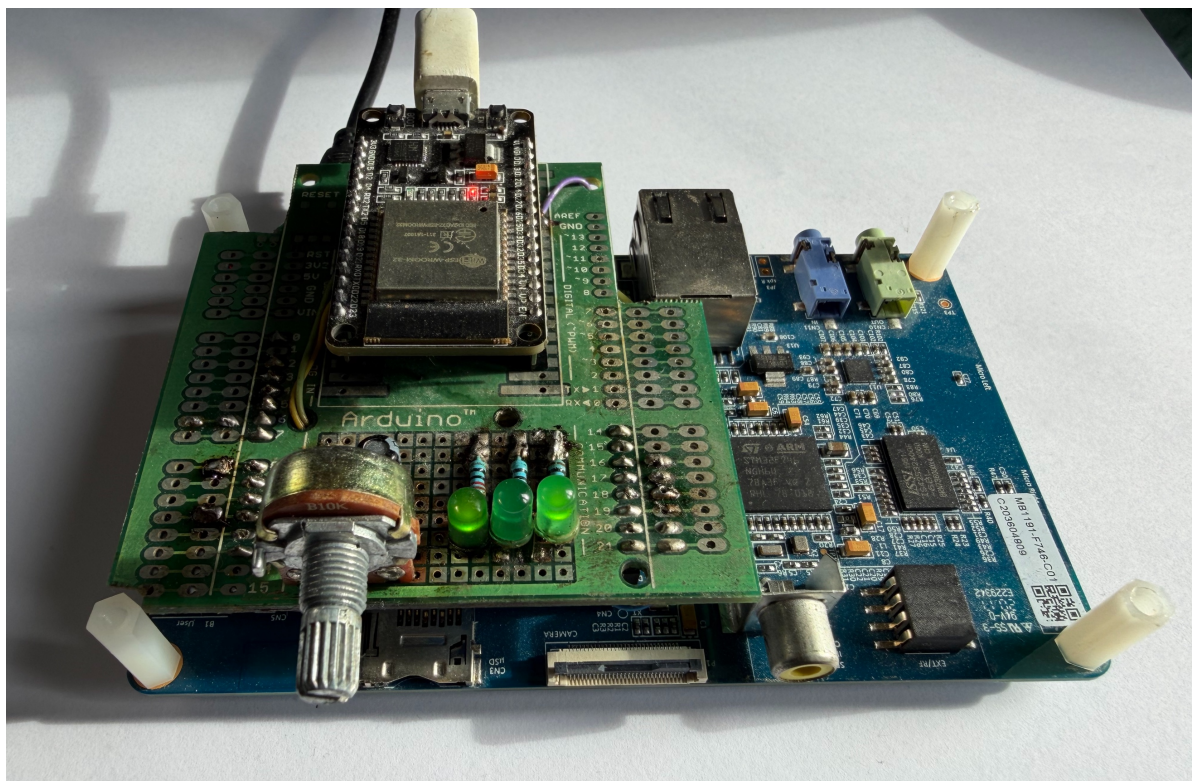
Tento RC filtr je konkrétně tvořen sériovým rezistorem $R1$ ($1\text{ k}\Omega$) a paralelním polarizovaným kondenzátorem $C1$ ($10\text{ }\mu\text{F}$) zapojeným k systémové zemi (GND). Filtr úspěšně integruje digitální vysokofrekvenční PWM signál o amplitudě 3.3 V z výstupního pinu A3 na vyhlazené stejnosměrné analogové napětí, čímž plní funkci simulovaného digitálně-analogového převodníku (DAC).

Paralelně ke kondenzátoru $C1$ je jako zátěž připojen lineární potenciometr $R2$ ($10\text{ k}\Omega$), který zastává funkci nastavitelného napěťového děliče. Středový jezdec potenciometru je zapojen přímo do analogového vstupního pinu A0, což umožňuje manuální regulaci přiváděného analogového napětí do analogově-digitálního převodníku (ADC) v rozsahu od 0 V do maximálně přibližně 3.0 V . Horní limit napětí je dán dělicím poměrem zatíženého filtru. Schéma zapojení analogového subsystému je uvedeno na Schématu 2.

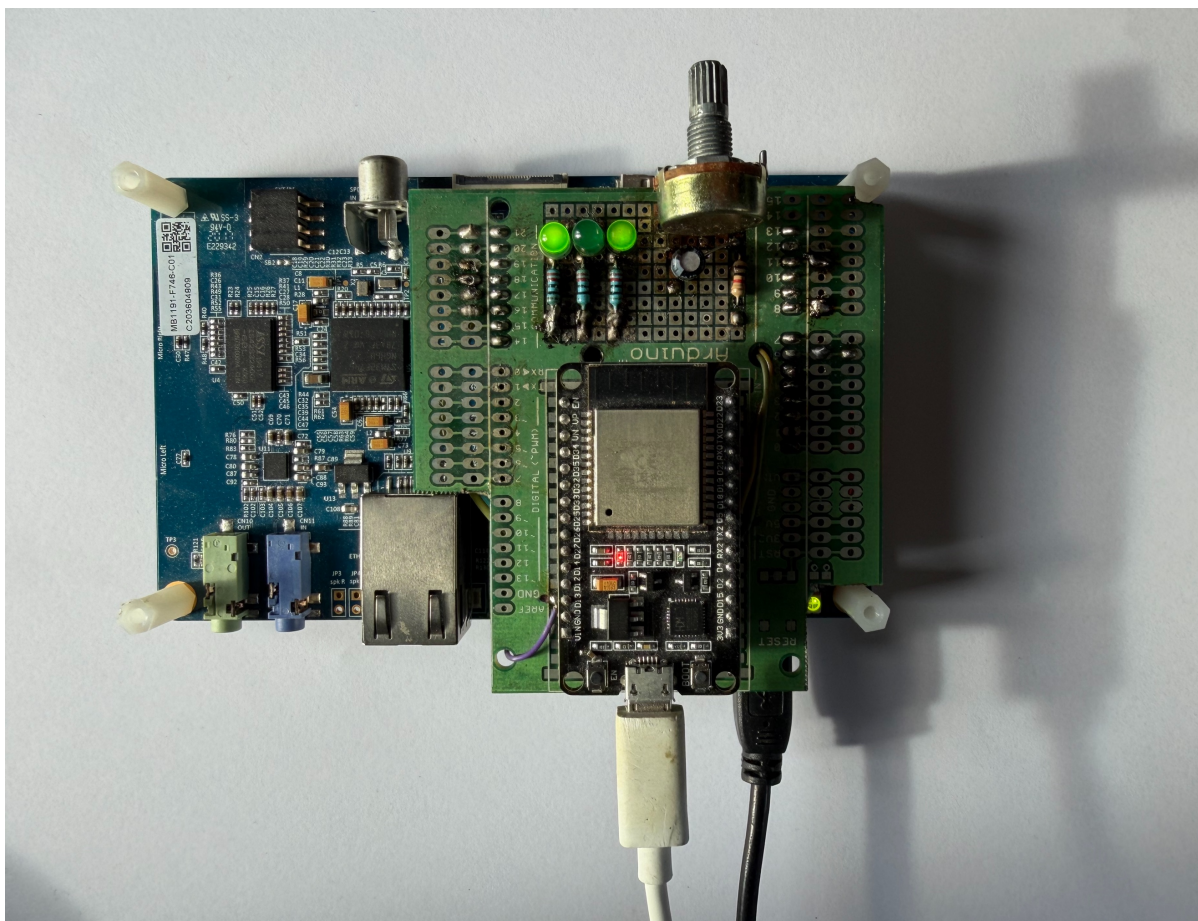
3.1.4 Finální hardwarové sestavení

Všechny výše popsané elektronické obvody byly fyzicky zkompletovány a spájeny na univerzální prototypové desce, která plní funkci rozšiřujícího shieldu. Tento shield je prostřednictvím standardizovaných konektorů kompatibilních s rozhraním ARDUINO® Uno V3 mechanicky a elektricky nasazen přímo na vývojovou desku STM32F746G-DISCO.

Modul ESP32 DEVKIT v1 je upevněn do připravené patice na tomtéž shieldu, čímž vznikl kompaktní, mechanicky odolný a snadno přenosný hardwarový celek. Reálné provedení, rozmístění součástek, indikačních LED diod a ovládacího potenciometru je zdokumentováno na fotografiích hotového systému na Obrázku 5 a Obrázku 6.



Obrázek 5 – Fotografie finálního zapojení systému – boční pohled na osazený shield s ESP32 a STM32 (vlastní zpracování)



Obrázek 6 – Fotografie finálního zapojení systému – celkový pohled shora na rozmístění indikačních a regulačních prvků (vlastní zpracování)

3.2 Návrh a struktura komunikačního protokolu

Pro zajištění spolehlivé, deterministické a efektivní výměny dat mezi mikrokontroléry platformy ESP a STM byl navržen vlastní aplikační protokol postavený na serializační knihovně a binárním formátu ProtocolBuffers s využitím syntaxe verze proto2. Volba tohoto formátu pro komunikaci typu MCU-to-MCU (Microcontroller-to-Microcontroller) byla motivována extrémně nízkou režijní náročností přenášených binárních dat, vysokou rychlostí serializačních algoritmů a možností oboustranné automatické generace nízkoúrovňového obslužného kódu v jazyce C. Použití prověřených překladačů (jako jsou protobuf-c a nanopb) zcela eliminuje riziko chyb, které běžně vznikají při manuální implementaci proprietárních stavových parserů a dekodérů.

3.2.1 Mechanismus dvoufázového přenosu dat

Z důvodu proměnlivé délky užitečných dat (payloadu) probíhá přenos každé zprávy po fyzické sériové lince UART ve dvou logických fázích. Zpráva je pro tento účel striktně rozdělena na fixní hlavičku FrameHeader a variabilní tělo FramePayload. Samotný proces odesílání a zapouzdření dat je realizován v následujících po sobě jdoucích krocích:

1. **Příprava aplikačního payloadu:** Data určená k odeslání, která mohou obsahovat jednoduché konfigurační instrukce, příkazy k nastavení stavu programů nebo komplexní vektory naměřených veličin, jsou nejprve naplněna do struktury a serializována do zprávy typu FramePayload.
2. **Výpočet přenosových parametrů:** Vzhledem k tomu, že různé typy zpráv mají odlišnou délku a vnitřní strukturu, je z hotového serializovaného bloku FramePayload exaktně spočítána jeho výsledná velikost v bajtech. Nad těmito binárními daty je zároveň vypočten kontrolní součet (CRC) pro ověření integrity.
3. **Generování a odeslání hlavičky:** Zjištěná velikost těla zprávy a hodnota vypočteného CRC kódu se vloží do struktury FrameHeader. Tato hlavička má konstantní velikost 10 bajtů, odesílá se do linky jako první a slouží příjemci jako indikátor toho, kolik bajtů dat bude bezprostředně následovat a jakým způsobem se má validovat jejich bezchybné doručení.
4. **Odeslání těla zprávy:** Ihned po odvysílání fixní hlavičky je do sériového rozhraní odeslán samotný proud serializovaných binárních dat zprávy FramePayload.

3.2.2 Řešené technické výzvy při asynchronním přenosu

Při návrhu softwarové vrstvy nad sériovým rozhraním UART bylo nutné ošetřit několik kritických problémů spojených s asynchronním charakterem přenosu dat:

- **Garance návaznosti rámců** (FrameHeader a FramePayload): V datovém toku musí hlavička a tělo zprávy bezprostředně následovat za sebou. Jakékoli narušení této sekvence (např. vlivem restartu jednoho z MCU) by vedlo k desynchronizaci příjemce.
- **Asynchronní příjem a neúplnost zpráv:** V okamžiku, kdy příjemce úspěšně zpracuje hlavičku a zná délku zprávy, nemusí být tělo (FramePayload) ještě fyzicky doručeno linkou. Softwarová obsluha na obou mikrokontrolérech je proto navržena jako neblokující a pracuje s postupným plněním přijímacího bufferu.
- **Detekce chyb a integrity přenosu:** Sériové linky UART jsou v reálném prostředí náchylné k elektromagnetickému rušení. Pokud interní algoritmus detekuje nesoulad mezi přijatými daty a hodnotou CRC kódu extrahovanou z hlavičky, jsou poškozená data okamžitě zahozena.
- **Optimalizace režie protokolu:** Časté odesílání malých balíčků sice snižuje latenci, ale výrazně zvyšuje režii linky kvůli konstantní velikosti hlavičky (10 bajtů). Protokol proto umožňuje agregaci dat (sdružování vzorků ADC/DAC do vektorů) pro dosažení optimálního poměru mezi vytížením sběrnice a aktualitou dat.

3.2.3 Definiční schéma zpráv

Níže je uveden zkrácený výtah stěžejních částí definičního souboru schématu zpráv, ze kterého kompilátor ProtocolBuffers generuje výsledné obslužné C knihovny pro obě mikrokontrolérové platformy. Samotný přenos zpráv po asynchronní sériové lince UART vyžadoval návrh specifické struktury datového rámce, který zajišťuje integritu dat. Navrhnul jsem si následující strukturu. Každá odesílaná zpráva se skládá ze dvou pevně navazujících částí:

- z fixní hlavičky (FrameHeader) a
- z proměnlivého užitečného zatížení (FramePayload).

Hlavička má konstantní velikost 10 bajtů a nese informaci o přesné délce nadcházející zprávy spolu s kontrolním součtem CRC, který slouží k detekci chyb a zahození poškozených dat.

V definičním schématu (Zdrojový kód 1) je jasné patrné rozdělení na fixní hlavičku s kontrolními mechanismy a flexibilní tělo využívající klíčové slovo *oneof* pro maximální úsporu alokované paměti RAM, což je u vestavěných systémů klíčový parametr.

```
1. // messenger version 0.2.1
2. syntax = "proto2";
3.
4. message FrameHeader {
5.   required fixed32 next_message_size = 1;
6.   required fixed32 crc = 2;
7. }
8.
9. message FramePayload {
10.  oneof payload {
11.    // 1. Test Int
12.    TestIntConfig test_int_config = 1;
13.    TestIntData test_int_data = 2;
14.
15.    // 2. Test Bandwidth
16.    TestBandwidthConfig test_bandwidth_config = 3;
17.    TestBandwidthData test_bandwidth_data = 4;
18.
19.    // 3. DAC/ADC Streaming
20.    StreamConfig stream_config = 5;
21.    StreamData stream_data = 6;
22.  }
23. }
24.
```

Zdrojový kód 1 – Definiční schéma komunikačního protokolu
v souboru *messenger.proto* (vlastní zpracování)

3.3 Softwarová implementace mikrokontroléru STM32

Aplikace pro mikrokontrolér STM32F746NG je navržena bez operačního systému jako bare-metal aplikace využívající architekturu supersmyčky (superloop) v kombinaci s deterministickou obsluhou hardwarových přerušení (ISR). Tento přístup zaručuje maximální výpočetní efektivitu, nízkou režii a přesné časování kritických operací. K hardwarové abstrakci a konfiguraci vnitřních periférií je využita knihovna STM32Cube HAL.

3.3.1 Hlavní programová smyčka a koordinace úloh

Běh programu v souboru *main.c* cyklicky koordinuje jednotlivé neblokující procesní rutiny. Celý životní cyklus aplikace se dělí na inicializační fázi a samotné periodické vykonávání úloh:

- **Inicializační fáze:** Po spuštění mikrokontroléru dochází k inicializaci systémové knihovny HAL, konfiguraci vnitřních zdrojů hodinových frekvencí a nastavení nezbytných GPIO pinů. Bezprostředně poté se spouští inicializační funkce aplikačního manažeru *ProgramMgr_Init* a inicializace sériového komunikačního kanálu *SerialLink_Init*.
- **Periodické vykonávání:** Nekonečná smyčka *while(true)* v periodických intervalech neblokujícím způsobem spouští obslužnou funkci sériové linky *SerialLink_Process* pro

příjem a odbavení příchozích zpráv z ESP32 a funkci *ProgramMgr_Process* zajišťující samotné řízení, generování a zpracování aktivních testovacích programů. Asynchronní události, jako je zachycení dat ze sériové linky, komunikují s hlavní smyčkou skrze systém stavových příznaků (flagů).

3.3.2 Řízení sériového rozhraní a zpracování rámců

Modul *serial_link.c* zajišťuje spolehlivý obousměrný přenos dat mezi oběma mikrokontroléry. Pro minimalizaci zátěže procesoru využívá rozhraní UART v kombinaci s přímým přístupem do paměti (DMA).

- **Příjem dat s detekcí nečinnosti:** Příjem je realizován asynchronně pomocí funkce *HAL_UARTEx_ReceiveToIdle_DMA* do cirkulárního bufferu o velikosti 8192 bajtů. K vyvolání přerušení dochází buď při plném zaplnění bufferu, nebo při detekci nečinnosti na lince (Idle Line), což radikálně snižuje frekvenci přerušení procesoru.
- **Cirkulární na lineární transformaci:** Při vyvolání události *SerialLink_RxEventCallback* jsou nově přijatá data z cirkulárního bufferu přesunuta do pomocného lineárního bufferu *process_buffer* pro následnou analýzu a deserializaci.
- **Resynchronizace při chybě:** Pokud hlavička přijatého rámce neodpovídá specifikaci protokolu (např. v důsledku šumu na sběrnici), ukazatel analýzy se v lineárním bufferu posune o pouhý 1 bajt vpřed a pokus o čtení se opakuje. Tento plovoucí mechanismus zajišťuje okamžité zotavení komunikace bez nutnosti resetu celého spojení.
- **Asynchronní odesílání:** Odesílání dat probíhá přes DMA řadič voláním *HAL_UART_Transmit_DMA*. Před zahájením nového přenosu funkce *SerialLink_SendPayload* neblokujícím dotazováním ověřuje stav připravenosti vysílače, čímž předchází kolizím na sběrnici.

3.3.3 Aplikační manažer

Modul *program_mgr.c* koordinuje spouštění a konfiguraci nezávislých měřicích a testovacích programů. Vzhledem k požadavkům na reálný čas a stabilitu přenosu řeší modul následující:

- **Synchronizace asynchronních toků:** Příchozí DAC vzorky přicházejí z ESP32 asynchronně v závislosti na zátěži Wi-Fi sítě. Pro zajištění plynulého a periodického výstupu na PWM implementuje modul kruhovou frontu typu FIFO o velikosti 8192 prvků, která vyrovnává časové rozdíly (jitter) přenosu.
- **Ochrana proti přetečení bufferu:** Pokud rychlost zápisu do FIFO překročí rychlost čtení, modul automaticky zahazuje nejstarší vzorky. Tím předchází zablokování systému a zachovává kontinuitu přehrávání novějších dat.
- **Škálování střidy PWM:** Vstupní 12bitové vzorky (rozsah 0 až 4095) musí být lineárně přepočítány na pracovní cyklus PWM, jehož celková perioda je řízena registrem Auto-Reload (ARR) časovače TIM13. Modul za běhu provádí tento přepočet podle vztahu:

$$pwm_val = \frac{dac_val \cdot (current_arr + 1)}{4096}$$

To zajišťuje správný poměr výstupního napětí nezávisle na dané frekvenci časovače.

- **Dávkování přenosu dat:** Odesílání jednotlivých ADC vzorků by zahltilo sběrnici UART režijními bajty protokolu. Modul proto naměřené vzorky shromažďuje v lokálním poli a odesílá je dávkově až po naplnění definované velikosti rámu (`samples_per_frame`), čímž maximalizuje propustnost linky.

3.4 Softwarová implementace modulu ESP32

Software pro mikrokontrolér ESP32 je implementován jako firmware v jazyce C/C++ s využitím oficiálního frameworku ESP-IDF a operačního systému reálného času FreeRTOS. Role tohoto modulu spočívá v řízení asynchronní síťové komunikace, obsluze webového serveru a zprostředkování datového mostu na sériovou linku UART.

3.4.1 Inicializace systému a správa Wi-Fi konektivity

Při startu mikrokontroléru dochází k inicializaci síťového rozhraní a spuštění stavového automatu Wi-Fi modulu. Systém se prioritně pokouší o připojení k lokální bezdrátové síti (režim Wi-Fi stanice) pomocí předdefinovaných přihlašovacích údajů. Pokud se modulu nepodaří navázat spojení ani po třech po sobě jdoucích pokusech, automaticky aktivuje nouzový mechanismus (Fallback AP) a přepne se do režimu přístupového bodu. Pro úsporu systémových prostředků a paměti RAM je v režimu AP nastaven striktní limit pro současné připojení maximálně čtyř uživatelů.

Reálná konfigurace parametrů Wi-Fi subsystému v rámci souboru *sdkconfig* je uvedena níže:

```

1. #
2. # Application - Wificonfiguration
3. #
4. CONFIG_WIFI_SEARCH_SSID="SecretWiFi"
5. CONFIG_WIFI_SEARCH_PASSWORD="secretpass"
6. CONFIG_WIFI_SEARCH_RETRIES=3
7.
8. # end of Application - Wificonfiguration
9. #
10. # Application - Wififallback AP configuration
11. #
12. CONFIG_WIFI_FB_AP_SSID="SecretWiFiESP"
13. CONFIG_WIFI_FB_AP_PASSWORD="secretpass"
14. CONFIG_WIFI_FB_AP_CHANNEL=1
15. CONFIG_WIFI_FB_AP_CONN=4
16. # end of Application - Wififallback AP configuration

```

Zdrojový kód 2 – Konfigurace Wi-Fi rozhraní v ESP-IDF projektu
(vlastní zpracování)

3.4.2 Sériová komunikace, asynchronní příjem a validace

Komunikační rozhraní na straně ESP32 zajišťuje asynchronní příjem, nízkoúrovňovou validaci datových rámců a jejich rekonstrukci do původních programových struktur.

- **Asynchronní příjem:** Příjem dat z UART linky je realizován neblokujícím způsobem na pozadí operačního systému pomocí vyhrazené FreeRTOS úlohy *rx_task*. Tato úloha přímo spolupracuje s přijímacím kruhovým bufferem o celkové velikosti 4096 bajtů. Úloha nejprve vyčítá bajty odpovídající fixní velikosti struktury *FrameHeader*. Jakmile

je hlavička úspěšně dekodována, systém extrahuje délku zprávy z pole *next_message_size* a vyčká na příchod kompletního těla zprávy *FramePayload*.

- **Resynchronizace při chybě:** Pokud se hlavičku nepodaří rozbalit (např. z důvodu poškození dat šumem), ukazatel v kruhovém bufferu se posune o pouhý 1 bajt a proces hledání začátku rámce se opakuje. Tím je zajištěno opětovné zachycení synchronizace bez nutnosti úplného resetu spojení.
- **Validace integrity dat:** Před samotným klientským zpracováním a deserializací těla zprávy probíhá hardwarová kontrola integrity. Ze sériových dat v payloadu je vypočten kontrolní součet pomocí hardwarově akcelerované funkce *esp_rom_crc32_le*. Hodnota je následně porovnána s polem *crc* v přijaté hlavičce. V případě jakékoli neshody je celý rámec okamžitě zahozen, což spolehlivě filtruje poškozené přenosy na sběrnici.
- **Správa paměti a předcházení únikům:** Vzhledem k dynamické alokaci paměti při rozbalování zpráv knihovnou *protobuf-c* byla implementována automatická správa zdrojů. Byl definován makro-identifikátor `__AUTO_FREE_MSG__` využívající kompilátorový atribut *cleanup* v jazyce C. Lokálně deklarované ukazatele na zprávy označené tímto atributem automaticky vyvolají uvolňovací funkci *protobuf_c_message_free_unpacked* při opuštění bloku kódu (scope), což eliminuje riziko fragmentace paměti RAM v přijímací smyčce.

3.4.3 Webový server pro vzdálené řízení systému

Pro zajištění uživatelského rozhraní a vzdáleného monitorování byl na platformě ESP32 implementován webový server. Využívá nativní komponentu *esp_http_server* z frameworku ESP-IDF a běží jako samostatná systémová úloha v operačním systému FreeRTOS.

Z důvodu limitovaných prostředků mikrokontroléru má server vypnutou podporu persistentních spojení (Keep-Alive), má nastaveny agresivní timeouty na hodnotu 2 sekund a využívá aktivní LRU (Least Recently Used) mechanismus, který při nedostatku soketů automaticky uzavírá nejstarší neaktivní připojení. Statické soubory uživatelského rozhraní (HTML/JS) jsou kompilovány a uloženy přímo do Flash paměti mikrokontroléru, což šetří drahocennou paměť RAM a zjednodušuje nasazení zařízení v terénu.

Přístup k připojeným klientům je chráněn mutexem ze systému FreeRTOS. Pro zápis na sériovou linku UART je zaveden exkluzivní zámek odesílatele, což znamená, že data na UART může posílat vždy pouze jeden aktivní WebSocket klient, zatímco požadavky ostatních souběžně připojených uživatelů jsou ignorovány, aby nedošlo k promíchání binárních dat. Server registruje sadu endpointů (API) zajišťujících kompletní ovládání systému, které byly detailně shrnuty v předchozí kapitole.

Pro zajištění kompletní modularity a snadné rozšiřitelnosti byl celý systém vzdálené správy striktně oddělen do logických endpointů. Kompletní přehled navrženého komunikačního rozhraní (API) integrovaného webového serveru, včetně metod a popisu nízkourovňového fungování jednotlivých cest, je přehledně shrnut v Tabulce 1.

URI cesta	Metoda	Popis a high-level fungování
/api/status	GET	Ověření dostupnosti serveru (vrací základní stavový JSON).
/api/program1	GET	Řízení základního testu (zapíná/vypíná indikační LED na STM32 a odesílá příkaz na UART).
/api/program2	GET	Konfigurace testu propustnosti UART linky s nastavením velikosti testovacího payloadu.
/api/program2stats	GET	Vrací živé statistiky (počet odeslaných a přijatých paketů) právě probíhajícího zátěžového testu.
/api/program3	GET	Aktivace streamování analogových dat (nastavení vzorkovací frekvence). Při vypnutí korektně odpojí WebSocket klienty.
/api/program3data	WebSocket	Duplexní datový kanál: Přenáší binární ADC vzorky z UARTu k webovému klientovi a asynchronně přijímá DAC vzorky zpět pro STM32.
* (wildcard)	GET	Zachytává ostatní požadavky a servíruje statický HTML/JS frontend přímo z interní Flash paměti mikrokontroléru.

Tabulka 1 – Přehled komunikačního rozhraní (API) na modulu ESP32 (vlastní zpracování)

Webová aplikace a uživatelské rozhraní

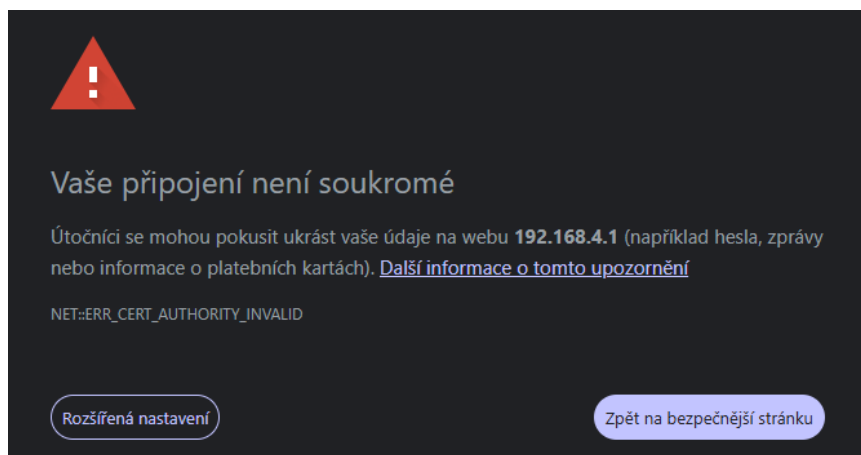
Uživatelské rozhraní celého systému je realizováno formou responzivní webové aplikace, kterou hostuje integrovaný webový server na modulu ESP32. Tato aplikace slouží jako grafická nadstavba (dashboard), která uživateli umožňuje plnou konfiguraci běžících podsystémů, volbu parametrů měření a vizualizaci datových proudů v reálném čase.

3.4.4 Síťová konektivita a inicializace připojení

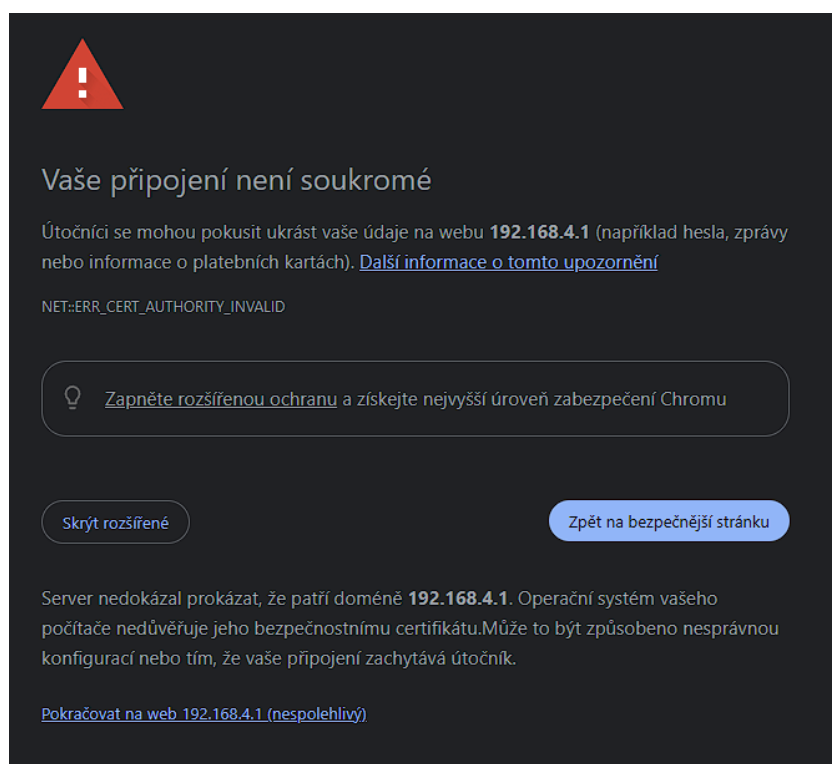
Způsob, jakým uživatel přistupuje k webovému rozhraní, závisí na aktuálním provozním režimu Wi-Fi modulu ESP32. Pokud se modul úspěšně připojí k lokální bezdrátové síti, očekává přidělení IP adresy z DHCP serveru. V opačném případě se modul přepne do režimu přístupového bodu (Fallback AP), kde uživatel zadá adresu výchozí brány *http://192.168.4.1/*.

Vzhledem k tomu, že vestavěný server komunikuje přes nešifrovaný protokol HTTP, moderní webové prohlížeče v zájmu bezpečnosti toto spojení blokují a uživateli zobrazí varování o

nezabezpečeném připojení. Tento stav a postup jeho manuálního zprovoznění přes pokročilé nastavení v prohlížeči ilustrují Obrázek 7 a Obrázek 8.



Obrázek 7 – Bezpečnostní upozornění webového prohlížeče při přístupu na lokální IP adresu



Obrázek 8 – Postup pro vynucení pokračování na webové rozhraní v pokročilém nastavení

3.4.5 Kombinovaný komunikační model rozhraní

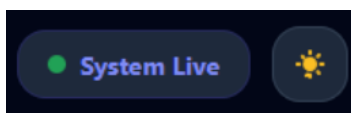
Aby byla zajištěna vysoká stabilita serveru a zároveň plynulý přenos dat, je komunikační vrstva webové aplikace navržena jako hybridní:

- **Bezstavová architektura REST (HTTP GET):** Tento model je využíván pro asynchronní zasílání jednorázových konfiguračních příkazů, jako je zapínání a vypínání jednotlivých demonstračních programů.

- **Plně duplexní streamování (WebSockets):** Pro úlohy vyžadující nepřetržitý přenos velkého objemu dat se otevírá trvalé WebSocket spojení, které eliminuje režii neustálého navazování HTTP relací.

3.4.6 Vizuální struktura a globální prvky rozhraní

Celkový vizuální návrh aplikace je podřízen požadavku na přehlednost a unifikované ovládání všech tří testovacích programů z jednoho místa. V pravé horní části aplikace se nacházejí dva globální prvky rozhraní zachycené na Obrázku 9. Prvním je panel System Status, který periodicky odesílá na pozadí požadavek *GET /api/status* a vizuálně uživateli signalizuje stav spojení s webovým serverem. Druhým prvkem je přepínač barevného tématu (Light/Dark mode).



Obrázek 9 – Indikátor aktivity systému a přepínač barevného tématu aplikace (vlastní zpracování)

Nedílnou součástí rozhraní je lokální log událostí nazvaný SystemConsole. Tento log zaznamenává akce provedené z hlediska síťové komunikace, jako jsou změny stavu programů, potvrzení o naplnění bufferu nebo případné odpojení soketů, jak ukazuje příklad reálného výpisu na Obrázku 10. Log je generován čistě na straně klientského prohlížeče.



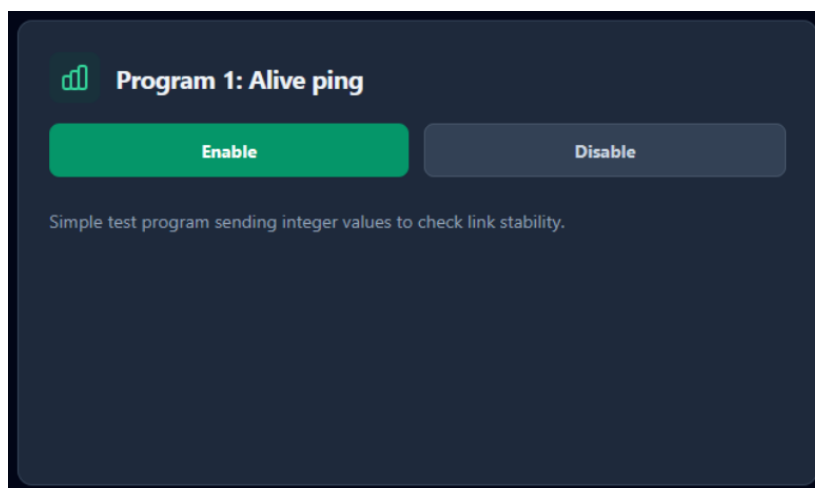
Obrázek 10 – Diagnostický log událostí (SystemConsole) na straně webového klienta (vlastní zpracování)

3.5 Ukázkové programy a testování propustnosti

Za účelem ověření funkčnosti navrženého komunikačního řetězce, otestování limitů přenosového protokolu a demonstrace spolupráce obou mikrokontrolérů v reálném čase byly implementovány tři samostatné programy.

3.5.1 Program 1: Verifikace komunikačního řetězce

První program ověřuje stabilitu celého přenosového subsystému a prověřuje, zda integrovaný webový server dokáže korektně transformovat uživatelské požadavky do binárních struktur protokolu ProtocolBuffers. Řízení programu je vyvedeno na dvě jednoduchá stavová tlačítka zobrazená na Obrázku 11.

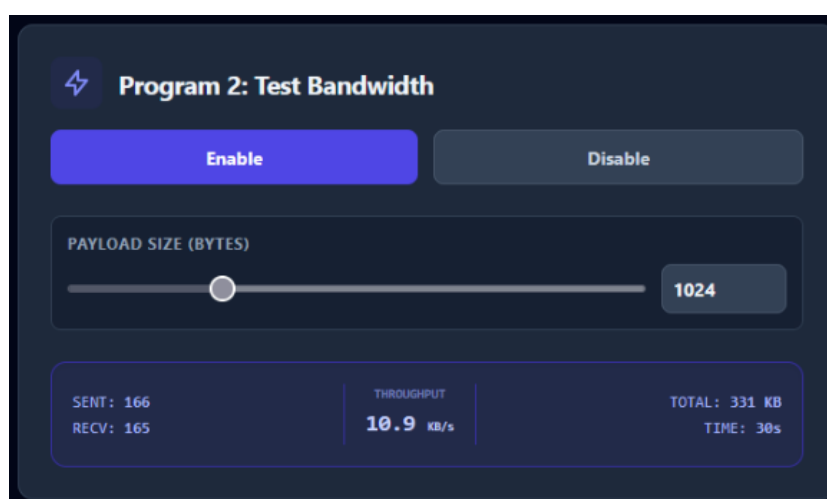


Obrázek 11 – Ovládací panel Programu 1 (Alive ping) ve webovém rozhraní (vlastní zpracování)

Požadavek je odesílán jako standardní REST volání na endpoint *GET/api/program1*. Vedle samotného spínání stavů je v tomto programu implementováno periodické vyčítání náhodného čísla generovaného přímo hardwarovým TRNG generátorem na desce STM32. Hodnota slouží jako indikátor responzivity linky a ověřuje, že hlavní supersmyčka na STM32 není blokována ani při zatížení síťového rozhraní.

3.5.2 Program 2: Analýza propustnosti

Druhý program se zaměřuje na zátěžové testování komunikačního protokolu a zkoumá vliv režie serializace na výslednou propustnost linky UART. Uživatel v grafickém rozhraní (viz Obrázek 12) definuje pomocí posuvníku velikost testovacího payloadu. Po spuštění vzniká uzavřená smyčka (ping-pong test) běžící na maximální možnou rychlost sběrnice, přičemž klient statistiky vyčítá přes endpoint *GET/api/program2stats*.



Obrázek 12 – Ovládací rozhraní a tabulka živých statistik propustnosti v Programu 2 (vlastní zpracování)

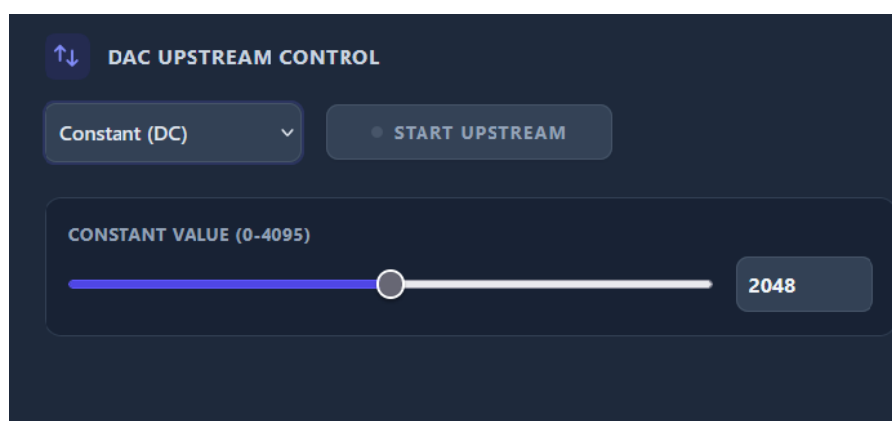
Měřením a analýzou tohoto testu bylo dosaženo klíčového zjištění o chování protokolu:

- **Payload menší než ~256 bajtů:** Projevuje se vysoká režie konstantní 10bajtové hlavičky. Úzkým hrdlem se stává samotný výpočetní výkon procesorů vyčleněný na nestálou serializaci a obsluhu rámců.
- **Payload větší než ~256 bajtů:** Výpočetní overhead se minimalizuje a data efektivně vyplní šířku pásma. Úzké hrdlo se přesouvá na hardwarovou úroveň, kde je limitujícím faktorem maximální přenosová rychlost (baud rate) samotného fyzického rozhraní UART.

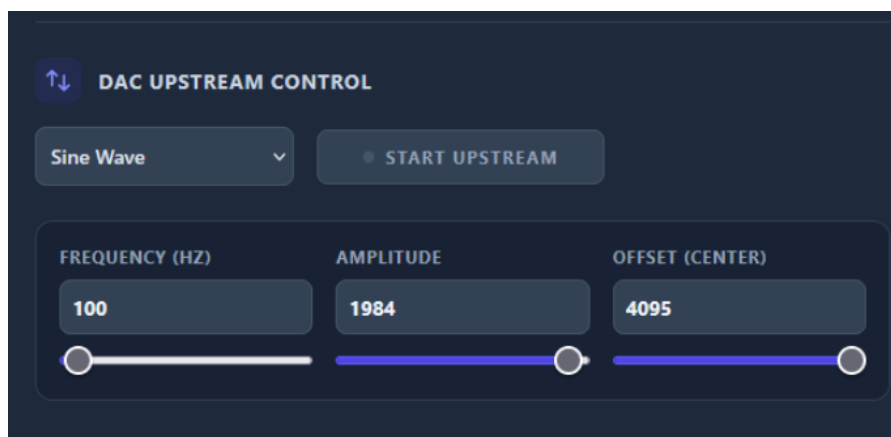
3.5.3 Program 3: Komplexní integrace

Tento program plně integruje softwarový komunikační řetězec s externím obvodem regulované analogové smyčky. Sběr, digitalizace a následný přenos dat z měřicích čidel tvoří elementární stavební kámen každého moderního vestavěného systému. Jak ve své práci zdůrazňuje (Novák, 2005), senzorický subsystém je v reálné technické praxi zodpovědný za spolehlivé mapování stavu okolního prostředí, což vyžaduje přesné a kontinuální vyhodnocování signálů. Program 3 v této práci slouží jako praktická demonstrace takového komplexního subsystému.

V rámci tohoto testovacího programu má uživatel možnost softwarově generovat signály na straně webové aplikace a následně v reálném čase sledovat jejich zpětné digitální vzorkování na obrazovce klientské aplikace. Před spuštěním samotného streamu uživatel volí typ požadovaného signálu, přičemž grafické rozhraní nabízí konstantní stejnosměrné napětí (Obrázek 13) nebo čistou sinusoidu s nastavitelnou frekvencí a amplitudou (Obrázek 14).

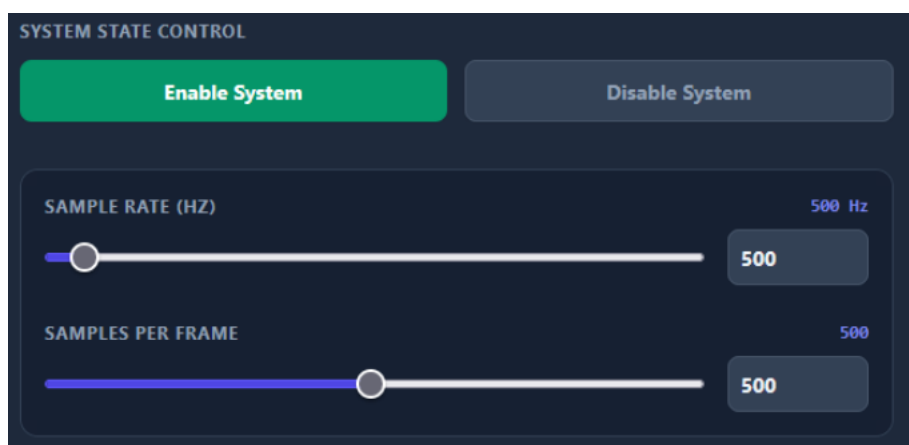


Obrázek 13 – Volba konstantního DC signálu (vlastní zpracování)



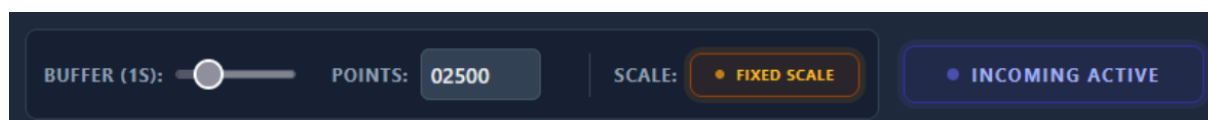
Obrázek 14 – Konfigurace parametrů sinusového signálu (vlastní zpracování)

Z hlediska optimalizace výkonu a balancování zátěže linky jsou klíčovými prvky dva posuvníky v konfiguračním panelu zobrazeném na Obrázku 15. Parametr Sample Rate určuje hardwarovou frekvenci vzorkování ADC převodníku na desce STM32, zatímco Samples per frame definuje, po kolika nasbíraných vzorcích má mikrokontrolér zabalit data do jedné zprávy Protobuf a odeslat ji na UART. Sdružování vzorků do větších rámců radikálně šetří systémové prostředky.



Obrázek 15 – Nastavení vzorkování a velikosti rámce (vlastní zpracování)

Přijímaná data se na frontendové straně vykreslují do kontinuálního grafu, jehož chování a plynulost lze ovlivnit parametry Buffer, Points (časové okno) a Scale na ovládací liště vyobrazené na Obrázku 16.



Obrázek 16 – Ovládací prvky pro zobrazení grafu (vlastní zpracování)

Reálný průběh měření sinusového signálu, který jasně zachycuje šum v datech a uživatelské zásahy do regulačního potenciometru (pokles a nárůst střední hodnoty napětí), je detailně zdokumentován na výsledném grafu na Obrázku 17.



Obrázek 17 – Reálná vizualizace streamovaných ADC dat v reálném čase se zachycením manuální regulace potenciometru (vlastní zpracování)

4 Diskuze

Cílem této kapitoly je kriticky zhodnotit zvolená technologická řešení, analyzovat neočekávané chybové stavy z průběhu integrace a definovat limity navrženého distribuovaného systému. Zpětná reflexe je klíčová pro pochopení chování embedded platform v reálných podmínkách a pro návrh budoucích optimalizací.

4.1 Překonané výzvy

Vývoj distribuovaného řetězce kombinujícího bare-metal aplikaci na STM32 s operačním systémem FreeRTOS na ESP32 přinesl řadu technických výzev na softwarové i hardwarové úrovni.

4.1.1 Diagnostika síťové vrstvy a limity HTTP/1.1

Při pokusech o plynulé streamování ADC dat z STM32 k webovému klientovi se projevil limit bezstavového protokolu *HTTP/1.1*, na němž server původně běžel. Protokol nepodporuje moderní streamovací nástroje (např. Stream API), což způsobovalo netransparentnost při výskytu chyb. Při výpadcích datového toku nebylo z chybových příznaků jasné, zda anomálie nastala při serializaci na UART lince, v bufferech ESP32, nebo až v klientském JavaScriptu, což značně komplikovalo nalezení příčiny. Problém vyřešil přechod na plně duplexní protokol WebSockets, který datový stream striktně oddělil od konfiguračního REST API.

4.1.2 Správa prostředků a úniky WebSocketů na mobilních zařízeních

Mikrokontroléry disponují limitovanými zdroji, a proto neuvolňování paměti v chybových stavech způsobuje kritické anomálie. V praxi docházelo k únikům a ztrátám WebSocket spojení u mobilních telefonů, pokud uživatel prohlížeč nečekaně uspal nebo ztratil Wi-Fi signál. Sokety zůstávaly na straně ESP32 otevřené, což vedlo k vyčerpání RAM a poolu volných soketů, takže server následně odmítal obsloužit nové klienty. Tuto zranitelnost vyřešilo zavedení agresivních dvousekundových timeoutů a implementace LRU mechanismu, který nuceně uzavírá nejstarší neaktivní spojení.

4.1.3 Hardwarové interference nepájených spojů

Nepájené kontaktní pole (breadboard) je vysoce efektivní pro prototypování, avšak mechanické spoje nejsou pevné a generují signálový šum. Během ožívování analogové smyčky měl omezovací rezistor s tenkými vodiči skrytý špatný kontakt uvnitř pole. Obvod přesto generoval výstupní hodnoty natolik blízké očekávaným datům, že fyzické zapojení nebylo okamžitě podezřelé. Chyba byla dlouho mylně diagnostikována v softwaru (v konfiguraci šířky pulzů PWM, ADC či časování na STM32). Problém definitivně odstranil až přechod na pevně spájený plošný spoj rozšiřujícího shieldu.

4.2 Možnosti budoucího rozšíření

Navržený distribuovaný systém vykazuje vysokou míru modularity, což otevírá široké spektrum možností pro jeho budoucí softwarové i hardwarové rozšíření v navazujících projektech.

- A. **Využití lokálního grafického rozhraní:** Vývojová deska STM32F746NG disponuje integrovaným LCD-TFT displejem s kapacitní dotykovou vrstvou. Tato periferie by mohla být implementována jako sekundární konfigurační nástroj přímo na zařízení, případně by dotyková plocha mohla sloužit jako interaktivní zdroj souřadnicových dat pro testování přenosové linky.
- B. **Zpracování reálných audio signálů:** Namísto generování syntetických dat (konstanty, sinusovky) z webového klienta se jako logický krok nabízí využití osazených 3,5 mm audio jack konektorů na desce STM32. Tyto vstupy by umožnily přímý sběr a digitalizaci reálných akustických signálů z okolního prostředí, což by zvýšilo aplikační potenciál systému v oblasti zpracování signálů.
- C. **Dynamická rekonfigurace sériové linky:** Významným softwarovým vylepšením by byla implementace dynamicky nastavitelné přenosové rychlosti (baud rate) sběrnice UART. Tento parametr by byl konfigurovatelný uživatelem přímo z webové aplikace, což by umožnilo za běhu optimalizovat šířku pásma a latenci podle specifických nároků aktivního programu.
- D. **Pokročilá diagnostika systémových prostředků:** Do uživatelského rozhraní by bylo vhodné implementovat komplexnější analytické nástroje. Tyto volitelné funkce by uživateli umožnily v reálném čase monitorovat stav hardwarových zdrojů na obou zařízeních, zejména alokaci paměti RAM, fragmentaci haldy a vytížení jednotlivých procesorových jader.

5 Závěr

Předložený bakalářský projekt úspěšně splnil všechny stanovené cíle v oblasti návrhu a realizace distribuovaného embedded systému. Propojením platform STM32F a ESP32 skrze sériové rozhraní UART se podařilo vytvořit kooperativní celek, ve kterém je nízkoúrovňové řízení hardwarových úloh v reálném čase striktně odděleno od asynchronní zátěže spojené se sítíovou komunikací a obsluhou webového uživatelského rozhraní.

Praktické testování potvrdilo připravenost obou zvolených hardwarových komponent. Bezdrátový modul ESP32 bezproblémově zvládá plynulou obsluhu Wi-Fi konektivity a stabilní asynchronní komunikaci s více připojenými klienty současně, což plně obhájí důvod jeho integrace do systému. Stejně tak mikrokontrolér STM32 vykazuje spolehlivé chování při generování a zpracování analogových signálů v obvodu regulované zpětné smyčky, přičemž jeho hardwarové limity jsou pro dané využití zcela dostačující.

Zátěžová analýza komunikačního řetězce ukázala, že hlavním úzkým hrdlem celého systému je samotná sériová linka UART, což má dvě odlišné příčiny. V případě požadavku na odesílání velkého množství malých samostatných zpráv se projevuje režijní overhead zvoleného protokolu ProtocolBuffers. Ačkoliv je binární serializace a deserializace dat rychlá, její procesní režie není „zadarmo“; neustálé generování požadavků na zpracování rámců vyvolává vysokou výpočetní zátěž a nenechává ostatní části systému optimálně pracovat.

Pokud se však přistoupí k odesílání dat po větších kusech a vzorky se sdružují do periodických dávek, tento softwarový problém s procesním overheadem zcela pomíjí. Úzkým bodem přenosu se v ten moment stává čistě hardwarová úroveň, tedy maximální přenosová rychlost (baud rate) samotného fyzického rozhraní UART. Dosažené výsledky tak prokazují, že navržená architektura je vysoce funkční, avšak vyžaduje precizní softwarové vybalancování velikosti datových rámců vůči propustnosti fyzické sběrnice.

Použitá literatura

Novák, Petr. 2005. *Mobilní roboty: pohony, senzory, řízení. 1. díl. Robotika.* Praha : BEN - technická literatura, 2005. ISBN 8073001411.

Espressif Systems. 2025. ESP32-WROOM-32 Datasheet. *Espressif Systems Centralized Documentation Platform (CDP).* [Online] 2025. [Citace: 05. 05 2026.] https://documentation.espressif.com/esp32-wroom-32_datasheet_en.pdf.

STMicroelectronics. 2026a. 32F746GDISCOVERY Data brief. *ST.* [Online] 04 2026a. [Citace: 10. 05 2026.] https://www.st.com/resource/en/data_brief/32f746gdiscovery.pdf.

—, **2026b.** UM1907 User manual: Discovery kit with STM32F746NG MCU. *STMicroelectronics.* [Online] 05 2026b. [Citace: 10. 05 2026.] https://www.st.com/resource/en/user_manual/um1907-discovery-kit-for-stm32f7-series-with-stm32f746ng-mcu-stmicroelectronics.pdf.

Espressif Systems. 2026. ESP-Dev-Kits Documentation: ESP32. *Espressif Systems.* [Online] 19. 05 2026. [Citace: 20. 05 2026.] <https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32/esp-dev-kits-en-master-esp32.pdf>.

TRONIXLAB. 2020. DOIT_ESP32_DevKit-v1_30P. *GitHub.* [Online] 2020. [Citace: 25. 04 2026.] https://github.com/TronixLab/DOIT_ESP32_DevKit-v1_30P?tab=readme-ov-file.

Mohanan, Vishnu. 2022. DOIT ESP32 DevKit V1 Wi-Fi Development Board – Pinout Diagram & Arduino Reference. *CIRCUITSTATE Electronics.* [Online] 20. 12 2022. [Citace: 10. 04 2026.] <https://www.circuitstate.com/pinouts/doit-esp32-devkit-v1-wifi-development-board-pinout-diagram-and-reference/>.

STMICROELECTRONICS. 2026c. STM32CubeIDE: Integrated Development Environment for STM32. *STMicroelectronics.* [Online] 2026c. [Citace: 18. 04 2026.] <https://www.st.com/en/development-tools/stm32cubeide.html>.

Espressif Systems. 2026d. ESP-IDF (Extension for Visual Studio Code). *Visual Studio Marketplace.* [Online] 2026d. [Citace: 26. 04 2026.] <https://marketplace.visualstudio.com/items?itemName=espressif.esp-idf-extension>.

MICROSOFT. 2026a. Dev Containers (v. 0.460.0). *Visual Studio Marketplace.* [Online] 05 2026a. [Citace: 10. 05 2026.] <https://marketplace.visualstudio.com/items?itemName=ms-vscode-remote.remote-containers>.

—, **2026b.** Visual Studio Code. *GitHub.* [Online] 2026b. <https://github.com/microsoft/vscode>.

Espressif Systems. 2026e. ESP-IDF Programming Guide: Get Started - ESP32. *Espressif Systems.* [Online] 2026e. [Citace: 16. 04 2026.] <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/get-started/index.html#software>.

Docker Inc. 2026. Docker overview. *Docker Documentation.* [Online] 2026. [Citace: 15. 04 2026.] <https://docs.docker.com/get-started/docker-overview/>.

PROTOBUF-C. 2026. Protocol Buffers implementation in C. *GitHub.* [Online] 2026. [Citace: 26. 04 2026.] <https://github.com/protobuf-c/protobuf-c>.

MICROSOFT. 2026c. Developing inside a Container. *Visual Studio Code.* [Online] 2026c. [Citace: 01. 05 2026.] <https://code.visualstudio.com/docs/devcontainers/containers>.

Mischianti, Renzo. 2021. DOIT ESP32 DEV KIT v1 high resolution pinout and specs. *Mischianti (mischianti.org).* [Online] 17. 02 2021. [Citace: 10. 03 2026.] <https://mischianti.org/doit-esp32-dev-kit-v1-high-resolution-pinout-and-specs/>.

Arm Ltd. ST-Discovery-F746NG. *Mbed OS*. [Online] [Citace: 18. 03 2026.] <https://os.mbed.com/platforms/ST-Discovery-F746NG/>.

Analog Devices. 2024. UART: A Hardware Communication Protocol. *Analog Dialogue*. [Online] 2024. [Citace: 16. 04 2026.] <https://www.analog.com/en/resources/analog-dialogue/articles/uart-a-hardware-communication-protocol.html>.

DeepBlueMbedded. 2024. How to Receive UART Serial Data with STM32 DMA / Interrupt / Polling. *DeepBlueMbedded*. [Online] 2024. [Citace: 13. 04 2026.] <https://deepbluembedded.com/how-to-receive-uart-serial-data-with-stm32-dma-interrupt-polling/>.

EmbeddedExpertIO. 2025. STM32 UART Part 2: Send Data using Interrupt and DMA. *EmbeddedExpertIO*. [Online] 2025. [Citace: 23. 04 2026.] <https://blog.embeddedexpert.io/?p=3645>.

Espressif Systems. 2021. FreeRTOS (ESP-IDF Programming Guide v4.2.1). *Espressif Systems Documentation*. [Online] 2021. [Citace: 12. 04 2026.] <https://www.google.com/search?q=https://docs.espressif.com/projects/esp-idf/en/v4.2.1/esp32/api-reference/system/freertos.html>.

Silicon Signals Pvt. Ltd. 2024. Selection of your embedded system's software architecture. *Medium*. [Online] 2024. [Citace: 12. 04 2026.] <https://medium.com/@siliconsignals.io/selection-of-your-embedded-systems-software-architecture-1ee4c296669f>.

GOOGLE PROTOBUF. 2026. Protocol Buffers. *Google / protobuf.dev*. [Online] 2026. [Citace: 17. 03 2026.] <https://protobuf.dev/>.

MDN Web Docs. 2025. The WebSocket API (WebSockets). *MDN Web Docs*. [Online] 2025. [Citace: 23. 04 2026.] https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API.

HWLab FIT ČVUT. 2016. Arduino shieldy. [Online] 03. 11 2016. [Citace: 24. 03 2026.] <https://hwlab.fit.cvut.cz/pripravky/mcu/arduino/shields>.

Přílohy

A Digitální archiv projektu

Nedílnou součástí tohoto bakalářského projektu je kompletní softwarová implementace pro obě mikrokontrolérové platformy (firmware pro STM32F746 v prostředí STM32CubeIDE a firmware pro ESP32 v prostředí ESP-IDF) spolu s responzivní klientskou webovou aplikací.

Všechny zdrojové kódy, definiční schémata binárního protokolu Protocol Buffers, konfigurační soubory Docker kontejneru pro unifikaci vývojového prostředí a doprovodná technická dokumentace jsou uloženy a verzovány v online repozitáři na platformě GitHub.

Tento digitální archiv slouží pro ověření funkčnosti, replikovatelnost celého distribuovaného komunikačního řetězce a je veřejně přístupný na následující internetové adrese:

URL repozitáře: <https://github.com/DanielKnourek/BProjekt>